



KIWIHACKS DOCUMENTATION

S.H.E.R.M.A.N PROJECT

SATELLITE HIGH ELEVATION RADAR MONITORING ARRANGEMENT NETWORK

IDEAS/START UP

S.H.E.R.M.A.N PROJECT

The S.H.E.R.M.A.N Project stands for Satellite High Elevation Radar Monitoring Arrangement Network. It focuses on satellite tracking and monitoring, a system designed to provide users an interactive view of not only satellite data but also the “space junk”. The project aims to make satellite information easier to access by combining satellite tracking, orbital trajectories, location data, and space debris monitoring into one platform. By using data from sources such as NASA and SpaceX, S.H.E.R.M.A.N will allow users to explore information about satellites and better understand their movements around the Earth.

GOD'S EYE VIEW

Our model was based on God's Eye which is a spy-satellite simulator within a browser with real sources which are public data, God's Eye View brings public signals into one explorable globe. Flight transponders, ship beacons, orbital elements, seismographs, and public cameras already tell us a lot about the world. God's Eye View puts them in the same place, so you can move between a global picture and individual aircraft, ship, or street. It runs locally in the browser, with the source code you can inspect and extend it.

SOURCES

API(CELESTRAK)

- [CelesTrak GP data formats](#)(Provides satellite orbital data used to calculate satellite positions and trajectories.)
- [Space-Track documentation](#)(Provides satellite records, historical orbital data and information about satellites re-entering Earth's atmosphere.)
- [N2YO API](#)(Tracks satellite positions and predicts when satellites will pass over a selected location.)
- [SatNOGS DB API](#)(Provides satellite images of Earth for monitoring land, oceans, the atmosphere and environmental changes.)
- [Earth Search root catalog](#)(Finds satellite images by location and date, including Sentinel-2 imagery.)
- [NASA CMR Search API](#)(Helps locate NASA Earth science datasets and satellite observation records.)
- [NASA FIRMS API](#)(Provides satellite-detected fire hotspots and their geographic locations.)
- ISS details- [CelesTrack\(ISS\(Zarya\)\)](#)



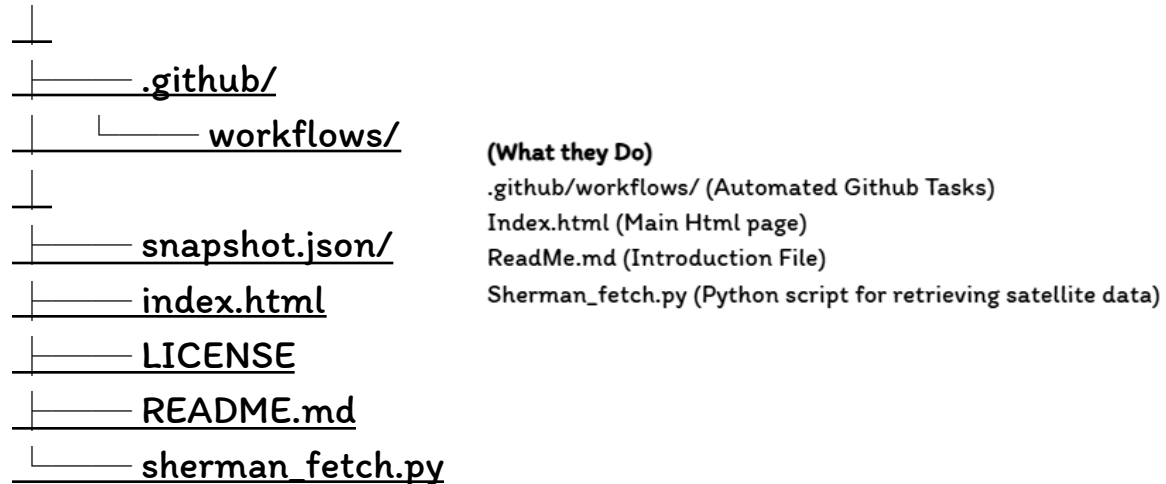
CELESTRAK[®]
SINCE 1985

CODE

STRUCTURE:

S.H.E.R.M.A.N

Satellite High Elevation and Radar Monitoring Arrangement Network



*We have used this structure to improve usability + readability for the collaborators of this project.

index.html(Main HTML Page)

This code creates the S.H.E.R.M.A.N satellite tracking website. It uses Cesium to show a 3D globe of Earth and JavaScript to make the website work. It tries to load satellite data from a file called snapshot.json. If the data is missing, it shows example satellites instead. Satellites are shown as coloured dots with names and lines showing their paths around Earth. Users can click on satellites to track them, hide or show different groups, and control the animation.

To visit the code please open: [Index.HTML](#)

*This was done due to the code being too long to produce in this google document.

sherman fetch.py(Python script used for retrieving satellite data)

```
import os, sys, json, time, urllib.parse, urllib.request, http.cookiejar
```

(Imports tools for files, data, time, web requests, and website login.)

```
BASE = "https://www.space-track.org"
```

(Stores the Space-Track website address.)

```
OUT = "snapshot.json"
```

(Sets the name of the file where satellite data will be saved.)

```
GP = "/basicspacedata/query/class/gp"
```

(Sets the address for requesting satellite orbital data.)

```
FIELDS = "/predicates/NORAD_CAT_ID,OBJECT_NAME,EPOCH,TLE_LINE1,TLE_LINE2/format/json"
```

(Selects the satellite ID, name, data time, and orbital information.)

```
PAUSE = 3
```

(Sets a three-second wait between requests.)

```
QUERIES = [
```

(Creates a list of satellite data requests.)

```
("NASA", GP + "/NORAD_CAT_ID/25544,20580,25994,27424,49260" + FIELDS),
```

(Gets orbital data for selected NASA-related satellites.)

```
("SpaceX", GP +  
"/OBJECT_NAME/STARLINK~~/EPOCH/%3Enow-3/orderby/NORAD_CAT_ID%20asc/limit/150" + FIELDS),
```

(Gets recent orbital data for up to 150 Starlink satellites.)

```
("SpaceX", GP + "/OBJECT_NAME/CREW%20DRAGON~~/EPOCH/%3Enow-10/limit/3" + FIELDS),
```

(Gets recent orbital data for up to three Crew Dragon objects.)

```
("Other", GP +  
"/OBJECT_TYPE/PAYLOAD/DECAY_DATE/null-val/EPOCH/%3Enow-3/orderby/NORAD_CAT_ID%20asc/limit/150" + FIELDS),
```

(Gets recent orbital data for other payloads that have not decayed.)

```
] + [
```

(Adds more requests to find space debris.)

```
("Junk", GP + "/OBJECT_NAME/" + urllib.parse.quote(n) + "~~/EPOCH/%3Enow-30/limit/15" + FIELDS)
```

(Gets orbital data for a selected type of space debris.)

```
for n in ("COSMOS 2251 DEB", "FENGYUN 1C DEB", "IRIDIUM 33 DEB")
```

(Names three groups of debris to search for.)

```
]
```

```
def main():
```

(Defines the main function that runs the program.)

```
user, pw = os.environ.get("SPACETRACK_USER"), os.environ.get("SPACETRACK_PASS")
```

(Gets the Space Track username and password from environment variables.)

```
if not user or not pw:
```

(Checks if the login details are missing.)

```
sys.exit("Set SPACETRACK_USER and SPACETRACK_PASS first.")
```

(Stop the program if the login details are missing.)

```
if os.path.exists(OUT) and "--force" not in sys.argv and time.time() - os.path.getmtime(OUT) < 4 * 3600:
```

(Checks if snapshot.json already exists and is less than four hours old, unless a forced update is requested.)

```
sys.exit("snapshot.json is under 4 hours old. Space-Track data updates no faster than that; use --force to override.")
```

(Stop the program to avoid downloading data again too soon.)

```
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(http.cookiejar.CookieJar()))
```

(Creates a web connection that can keep track of the login session.)

```
login = urllib.parse.urlencode({"identity": user, "password": pw}).encode()
```

(Prepares the login details for Space-Track.)

```
reply = opener.open(BASE + "/ajaxauth/login", login).read().decode()
```

(Sends the login details and reads the response.)

```
if "Failed" in reply:
```

(Checks if the login failed.)

```
sys.exit("Login failed. Check your username and password.")
```

(Stops the program if the login was unsuccessful.)

```
seen, objects = set(), []
```

(Creates a set to track duplicate satellites and a list to store the results.)

```
try:
```

(Starts a section that attempts to download the satellite data.)

```
for group, path in QUERIES:
```

(Goes through each satellite data request.)

```
try:
```

(Attempts to download the requested data.)

```
rows = json.loads(opener.open(BASE + path).read().decode())
```

(Downloads the results and converts the JSON data into Python data.)

```
except Exception as e:
```

(Catches errors that occur during a request.)

```
print("Query failed:", path[:80], e)
```

(Displays the failed request and its error.)

```
rows = []
```

(Uses an empty list if the request fails.)

```
for r in rows:
```

(Goes through each satellite in the results.)

```
if r.get("NORAD_CAT_ID") in seen or not r.get("TLE_LINE1"):
```

(Skips duplicate satellites and records without the required orbital data.)

```
continue
```

(Moves to the next satellite.)

```
seen.add(r["NORAD_CAT_ID"])
```

(Records the satellite ID to prevent duplicates.)

```
objects.append({"name": r["OBJECT_NAME"], "group": group, "norad": r["NORAD_CAT_ID"],  
               "epoch": r["EPOCH"], "line1": r["TLE_LINE1"], "line2": r["TLE_LINE2"]})
```

(Saves the satellite name, group, ID, data time, and orbital information.)

```
time.sleep(PAUSE)
```

(Waits three seconds before making the next request.)

```
finally:
```

(Runs the cleanup section even if a problem occurs.)

```
try:
```

(Attempts to log out of Space-Track.)

```
opener.open(BASE + "/ajaxauth/logout")
```

(Logs out of the Space Track account.)

```
except Exception:
```

(Ignores any logout errors.)

pass

(Continues the program without stopping.)

with open(OUT, "w") as f:

(Opens snapshot.json to save the satellite data.)

json.dump({"fetched": time.strftime("%Y-%m-%d %H:%M UTC", time.gmtime()), "objects": objects}, f)

(Saves the download time and satellite information as JSON.)

print(f"Saved {len(objects)} objects to {OUT}")

(Displays how many satellite records were saved.)

if __name__ == "__main__":

(Checks whether this file is being run directly.)

main()

(Runs the main function.)

Feedback

We used feedback from Kiera, Kyle, Christie and Maggie to help improve S.H.E.R.M.A.N. Keira supported the highlighting feature, while Kyle suggested adding a favourites list. Cristy and Maggie suggested adding a preview and cultural features. We used this feedback to identify useful features to develop and areas where the website could be improved.

Problems(Bugs) Checklist

1. UI Changes

Locator UI

- Fix the locator UI overlapping with the time controller. 
- Improve scrolling in the locator UI to make it easier to use. 

- Keep the S.H.E.R.M.A.N title visible when the locator UI is hidden. ✓
- Fix the Hide/Show button moving when clicked. ✓
- Make clicking “No Countries” deselect all countries. ✓

Time Controller

- Make the time controller display specific numerical values. ✓
- Increase the available simulation speed. ✓
- Use New Zealand Time (NZT). ✓
- Fix the pause button so it switches correctly between Play and Pause icons. ✓
- Add a border around the time controller UI. ✓

Information UI

- Make the X (close) button easier to click. ✓
- Add more satellite information, including longitude, latitude and velocity. ✓
- Make the X button stop satellite tracking when closing the information panel. ✓
- Add a button to move the information panel to the side of the screen. ✓
- Prevent the information panel from covering home buttons and other controls. ✓

2. Map and Website Improvements

- Fix satellite orbits so they display as complete circles where appropriate. ✓
- Add a loading screen and a reconnection screen. ✓
- Fix country borders and remove chunking or visual gaps. ✓
- Add an explanation of what S.H.E.R.M.A.N. is and what it does. ✓
- Incorporate the S.H.E.R.M.A.N. logo. ✓
- Improve the accuracy of the Arctic region. ✓
- Allow users to move the camera away from a satellite while tracking it. ✓
- Fix the map centring, particularly around Russia. ✓
- Ensure Fiji and Antarctica are not incorrectly marked as crash sites. ✓
- Improve zooming to allow users to view satellites in higher orbits. ✓
- Add dots or markers to S.H.E.R.M.A.N.'s visual design. ✓
- Fix the spelling of Eswatini. ✓
- Change the default home view so it does not centre on the United States. ✓

3. Code Changes

- Replace placeholder content with the appropriate data and functionality. ✓
- Fix satellites disappearing when tracking ends. ✓
- Remove the live video features. ✓
- Simplify and clean up the code where possible. ✓
- Review AI generated code and verify its accuracy. ✓
- Rewrite sections of code where necessary. ✓

- Conduct more extensive bug testing. ✓
- Implement satellite trajectory tracking. ✓

4. Camera Controls and Zooming

- Improve mouse controls to make zooming and movement easier. ✓
- Centre camera movement and zooming around the centre of the Earth. ✓
- Fix the globe and tilted map breaking when using the middle mouse button. ✓
- Fix unwanted zooming when holding and moving the middle mouse button without intending to zoom. ✓
- Improve middle mouse button zoom sensitivity and control. ✓
- Remove or repurpose the middle mouse button functionality within the controls if it cannot be made user-friendly. ✓
- Fix bugs when middle clicking and swiping upwards on the flat map. ✓

5. Satellite Tracking and Rendering

- Fix flickering affecting Vanguard 2 and Vanguard 3. ✓
- Fix satellite shaking or shuddering during rendering. ✓
- Investigate and fix other satellites experiencing similar issues to the Vanguard satellites. ✓
- Ensure satellite tracking and trajectory visualisation work smoothly. ✓
- Make the locator UI clearly identify which satellite or object the user has selected. ✓

AI Usage

Camera Fixes: We used AI to help fix camera movement and zooming problems.

Flickering: We used AI to help fix satellites flickering and shaking on the map.

General Bugs: We used AI to find bugs and help fix problems in the code.

Decorations: We used AI to make the website look better and more interesting.

Loading Screen: We used AI to help create a loading screen that appears while the website loads.

Random Selection Circles: We used AI to help add random circles to the map to make it more interactive.

*Note the use of AI was mainly to help identify bugs and suggest improvements.

LOGO/ART

Hex code's

Green #14FF00
 Blue #0040FF
 Cyan #00FFF7
 Red #FF0000
 Yellow #FFF500
 Magenta #9002FF
 Orange #FF5C00
 Black #000000
 Hot Pink #FF00A1
 Dark Blue #003F7F

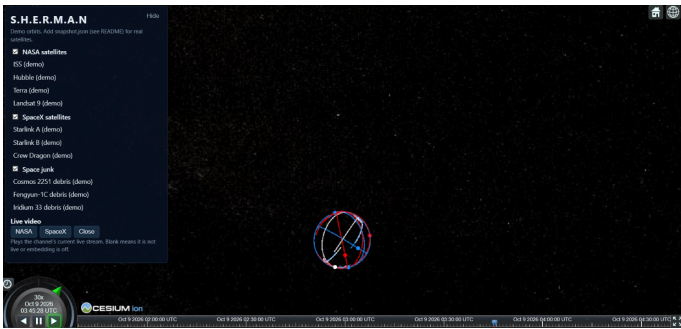


03 / GLITCH FRAME (FOR HERO, SOCIAL, POSTERS)



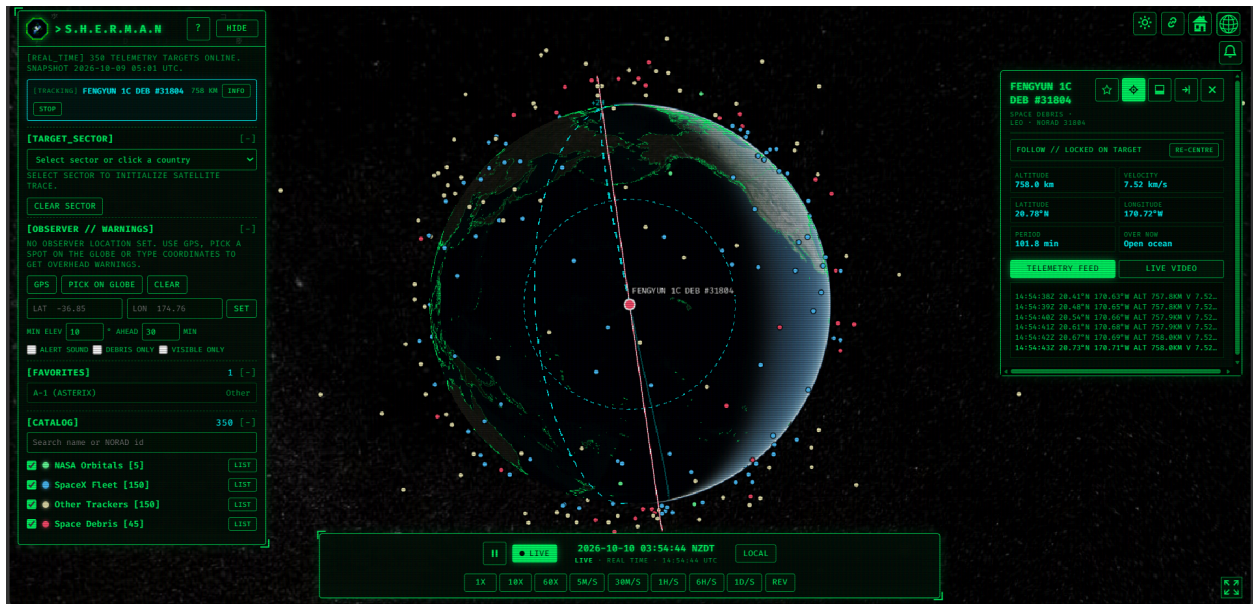


BEFORE AND AFTER



image





Congo

Cook Islands

Costa Rica

Côte d'Ivoire

Croatia

Cuba

Curaçao

Cyprus

Czechia

Democratic Republic of the Congo

Denmark

Djibouti

Dominica

Dominican Republic

Ecuador

Egypt

El Salvador

Equatorial Guinea

Eritrea

Estonia

Search name or NORAD id

☒ NASA Orbitals [5] LIST

AQUA

HST

ISS (ZARYA)

LANDSAT 9

TERRA

☒ SpaceX Fleet [150] LIST

☒ Other Trackers [150] LIST

☒ Space Debris [45] LIST